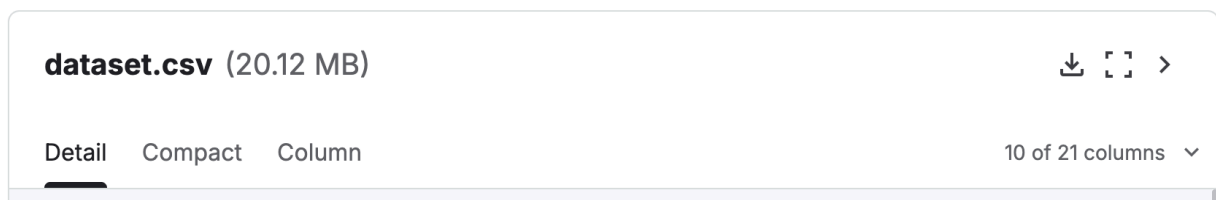


Final Project Report

Introduction

For this project, I chose a Spotify dataset containing **114,000** tracks spanning **114** distinct genres, with exactly **1,000** tracks per genre. Each track includes audio features such as danceability, energy, loudness, tempo, acousticness, valence, and speechiness, along with a popularity score (0–100) and an explicit content flag. I found this data interesting because Spotify algorithmically quantifies subjective musical qualities and assigns numerical scores to abstract metrics. This raises the question of whether popularity can truly be predicted from sound alone. My **hypothesis** going in was that **energy and loudness would be the strongest drivers of popularity, and that explicit tracks would tend to score higher**. However, even before diving into the analysis, a few observations caught my attention. The proportion of explicit tracks was much lower than I expected, and some genres I assumed would dominate in popularity based on energy alone did not seem to. Spotify also weights its popularity score toward recency, meaning a legendary older song could rank below a newer, less iconic one. This made me realize the analysis would not be as straightforward. These early contradictions made the dataset all the more compelling to explore.

To download the dataset, go to this link [📄 Spotify Tracks Dataset](#). You might need to sign in through a google or email account to access it. After that, just click the download button at the site directed.



Data Analysis Technique

To test this hypothesis, two data analysis techniques were applied. First, correlation analysis was used to measure the r value between each audio feature and popularity, quantifying how strongly each feature relates to popularity on a scale of -1 to $+1$. This directly addresses whether energy and loudness are the strongest drivers by comparing their r values against all other features.

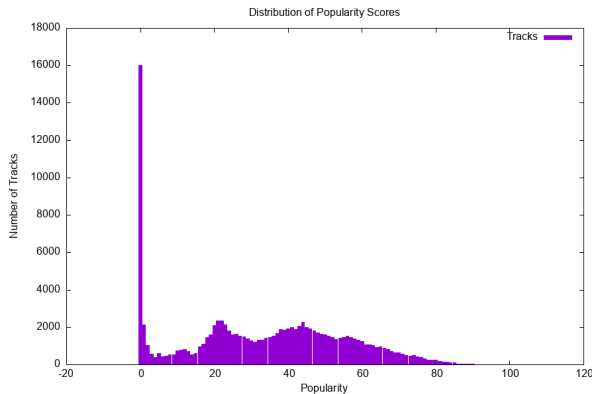
Second, linear regression was applied to model the relationship between energy and popularity, computing the best-fit line using the least squares method to find the slope and intercept that minimizes prediction error. The regression line makes it visually clear whether energy has any predictive power over popularity, and a predicted vs. actual plot was generated to evaluate how well the model fits the data.

Together, these two techniques provide both a broad overview of which features matter most and a focused test of the energy-popularity relationship at the center of the hypothesis.

Plots and Results

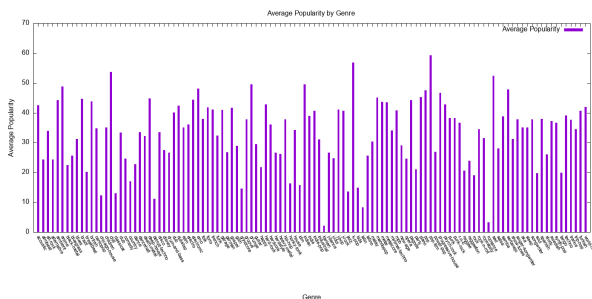
Plot	Result
------	--------

Distribution of popularity scores across the dataset



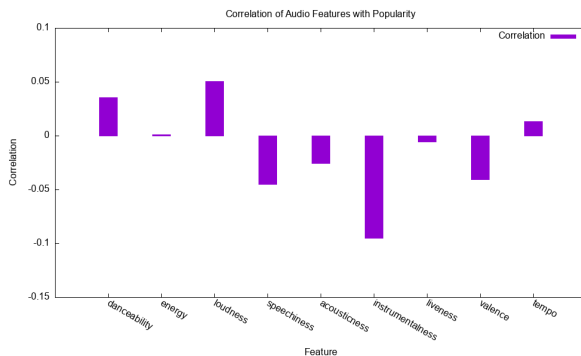
The distribution is right-skewed with a large spike at 0, indicating many obscure or unlisted tracks. The main cluster falls between 20–60, with very few tracks reaching above 80, suggesting that high popularity is rare across the dataset.

Average popularity by genre (bar chart)



Pop-film and k-pop show the highest average popularity scores, while genres like black-metal and classical rank near the bottom. Notably, high-energy genres do not dominate the top, which challenges the hypothesis that energy drives popularity.

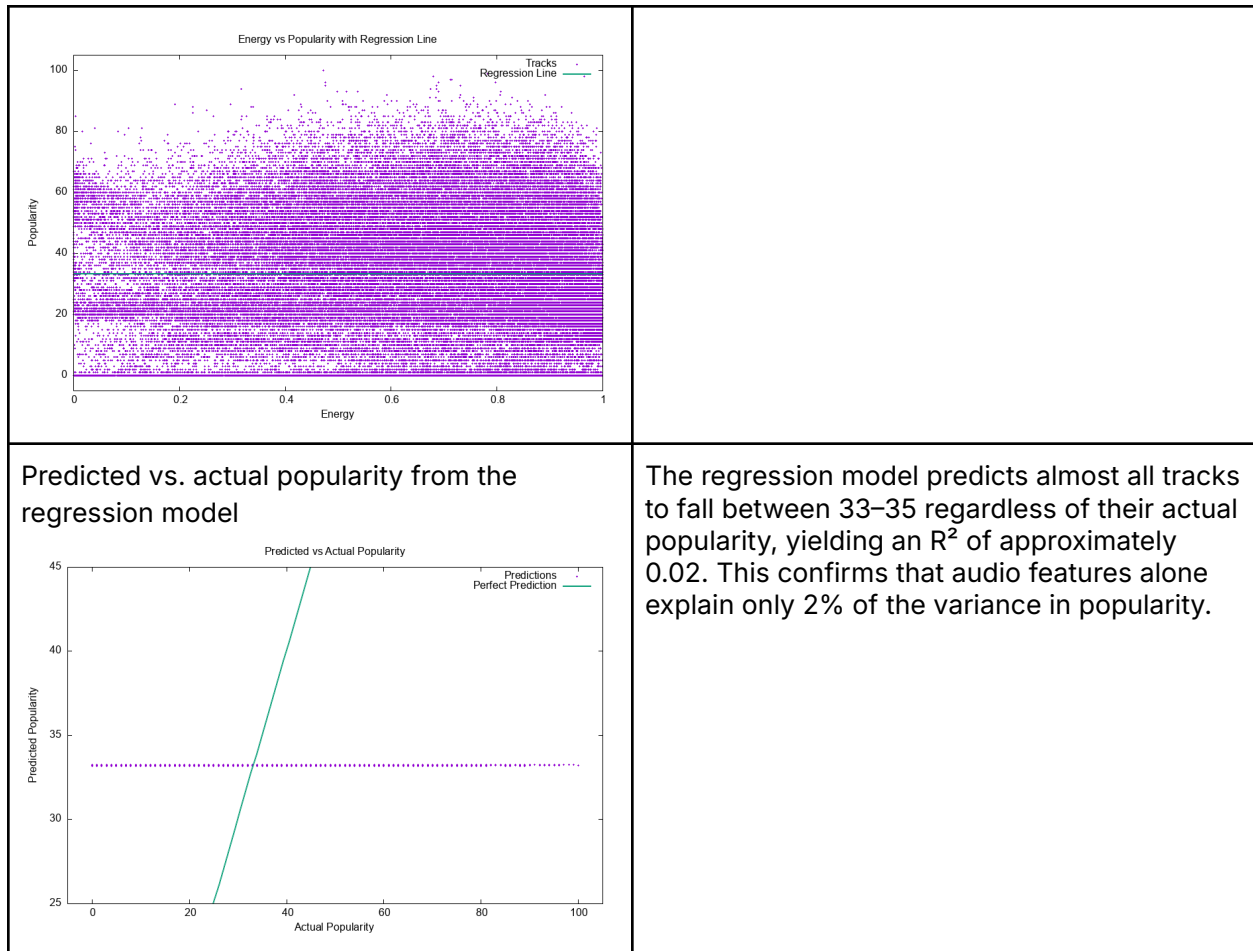
Correlation heatmap of audio features vs. popularity



All audio features show weak correlations with popularity. Instrumentalness has the strongest negative correlation ($r \approx -0.10$), suggesting tracks with vocals tend to be more popular. Energy has nearly zero correlation ($r \approx 0.001$), directly contradicting the hypothesis.

Scatter plot of energy vs. popularity with regression line

The slope of the regression line is approximately 0.09, which means every 1-unit increase in energy (the full range from 0 to 1), popularity only increases by 0.09 points on average. This confirms that energy has no meaningful predictive power over popularity. Tracks with high and low energy scores are equally spread across all popularity levels.



Code & Explanation

→ To compile: open the project in Xcode and press Cmd+R, or run `g++ main.cpp -o main && ./main` from the terminal in the project directory.

Code	Explanation
<pre> 8 #include <iostream> 9 #include <fstream> 10 #include <sstream> 11 #include <string> 12 #include <vector> 13 #include <map> 14 #include <cmath> 15 #include <algorithm> 16 #include <cstdlib> </pre>	Importing tools the program needs. Key libraries include <code>fstream</code> that reads files, <code>vector</code> that stores lists of tracks, <code>map</code> that groups data by genre, and <code>cmath</code> that provides <code>sqrt()</code> for correlation.

```

20 struct Track {
21     int index;
22     string track_id;
23     string artists;
24     string album_name;
25     string track_name;
26     int popularity;
27     long duration_ms;
28     bool explicit_content;
29     double danceability;
30     double energy;
31     int key;
32     double loudness;
33     int mode;
34     double speechiness;
35     double acousticness;
36     double instrumentalness;
37     double liveness;
38     double valence;
39     double tempo;
40     int time_signature;
41     string track_genre;
42 };

```

I created **struct Track** to group each track and relevant information. Each Track object holds one row of data with 21 columns from the CSV. Using the right types matters: int for whole numbers like popularity, double for decimals like energy (0.0–1.0), bool for true/false like explicit content.

```

44 vector<string> parseCSVLine(const string& line) {
45     vector<string> fields;
46     string field;
47     bool inQuotes = false;
48
49     for (char c : line) {
50         if (c == '"') {
51             inQuotes = !inQuotes;
52         } else if (c == ',' && !inQuotes) {
53             fields.push_back(field);
54             field.clear();
55         } else {
56             field += c;
57         }
58     }
59     fields.push_back(field);
60     return fields;
61 }

```

In this code snippet, I'm parsing the data. Functionally, the computer goes character by character through the line. When it sees a comma, it saves the current field and starts a new one. The inQuotes flag handles artist names like "Drake, Future" (commas inside quotes are part of the name), not separators. The result is stored in a **vector<string>** called fields. Each time a comma is found, **fields.push_back(field)** appends that field to the vector, growing it one entry at a time. By the end of the loop, fields hold one string per column in the exact order they appear in the CSV, which loadCSV then uses to assign values like f[5] → popularity and f[9] → energy into each Track struct.

```

63 vector<Track> loadCSV(const string& filename) {
64     vector<Track> tracks;
65     ifstream file(filename);
66
67     if (!file.is_open()) {
68         cerr << "Error: could not open file " << filename << endl;
69         return tracks;
70     }
71
72     string line;
73     getline(file, line); // skip header
74
75     while (getline(file, line)) {
76         auto f = parseCSVLine(line);
77         if (f.size() < 21) continue;
78
79         Track t;
80         t.index          = stoi(f[0]);
81         t.track_id       = f[1];
82         t.artists        = f[2];
83         t.album_name     = f[3];
84         t.track_name     = f[4];
85         t.popularity     = stoi(f[5]);
86         t.duration_ms    = stol(f[6]);
87         t.explicit_content = (f[7] == "True");
88         t.danceability   = stod(f[8]);
89         t.energy         = stod(f[9]);
90         t.key            = stoi(f[10]);
91         t.loudness       = stod(f[11]);
92         t.mode           = stoi(f[12]);
93         t.speechiness    = stod(f[13]);
94         t.acousticness   = stod(f[14]);
95
96         t.instrumentalness = stod(f[15]);
97         t.liveness        = stod(f[16]);
98         t.valence         = stod(f[17]);
99         t.tempo           = stod(f[18]);
100        t.time_signature  = stoi(f[19]);
101        t.track_genre     = f[20];
102
103        tracks.push_back(t);
104    }
105
106    return tracks;
107 }

```

Opens the CSV file, skips the header row, then loops through every remaining line. Each line gets parsed into fields, then assigned to a **Track struct**. **stoi()** converts strings to integers, **stod()** to decimals. Each completed track is added to the tracks vector which is returned at the end.

```

108 void writePopularityDistribution(const vector<Track>& tracks) {
109     ofstream out("popularity_distribution.dat");
110
111     map<int, int> counts;
112     for (const auto& t : tracks) {
113         counts[t.popularity]++;
114     }
115     for (const auto& p : counts) {
116         out << p.first << " " << p.second << "\n";
117     }
118     out.close();
119
120     ofstream gp("popularity_distribution.gp");
121     gp << "set terminal png size 900,600\n";
122     gp << "set output 'popularity_distribution.png'\n";
123     gp << "set title 'Distribution of Popularity Scores'\n";
124     gp << "set xlabel 'Popularity'\n";
125     gp << "set ylabel 'Number of Tracks'\n";
126     gp << "set boxwidth 0.8\n";
127     gp << "set style fill solid\n";
128     gp << "plot 'popularity_distribution.dat' using 1:2 with boxes title 'Tracks'\n";
129     gp.close();
130
131     system("/opt/homebrew/bin/gnuplot popularity_distribution.gp");
132 }

```

This function takes the full **tracks** vector and counts how many tracks have each popularity score using a `map<int, int>` called `counts`, where the key is the popularity score (0–100) and the value is how many tracks have that score. **counts[t.popularity]++** increments the count each time a score is seen. Once all tracks are counted, it writes the results to **popularity_distribution.dat**, one line per score in the format score count. It then writes a gnuplot script **popularity_distribution.gp** that configures the chart title, axis labels, box width and fill style, and tells gnuplot to plot the .dat file as a bar chart. Finally, `system()` runs gnuplot using the full path `/opt/homebrew/bin/gnuplot` to generate `popularity_distribution.png`.

```

134 void writeAveragePopularityByGenre(const vector<Track>& tracks) {
135     map<string, pair<double, int>> genreStats;
136
137     for (const auto& t : tracks) {
138         genreStats[t.track_genre].first += t.popularity;
139         genreStats[t.track_genre].second++;
140     }
141
142     ofstream out("avg_popularity_by_genre.dat");
143     for (const auto& p : genreStats) {
144         double avg = p.second.first / p.second.second;
145         out << "\"" << p.first << "\" " << avg << "\n";
146     }
147     out.close();
148
149     ofstream gp("avg_popularity_by_genre.gp");
150     gp << "set terminal png size 1400,700\n";
151     gp << "set output 'avg_popularity_by_genre.png'\n";
152     gp << "set title 'Average Popularity by Genre'\n";
153     gp << "set xlabel 'Genre'\n";
154     gp << "set ylabel 'Average Popularity'\n";
155     gp << "set style data histograms\n";
156     gp << "set style fill solid\n";
157     gp << "set xtics rotate by -60\n";
158     gp << "set xtics font ',8'\n";
159     gp << "set yrange [0:70]\n";
160     gp << "plot 'avg_popularity_by_genre.dat' using 2:xtic(1) title 'Average Popularity'\n";
161     gp.close();
162
163     system("/opt/homebrew/bin/gnuplot avg_popularity_by_genre.gp");
164 }

```

This function uses a **map<string, pair<double, int>>** called **genreStats** to accumulate the total popularity and track count for each genre. For every track, **genreStats[t.track_genre].first** adds the popularity score and **.second** increments the count. Once all tracks are processed, it loops through the map and divides the total popularity by the count to get the average for each genre, writing the result to **avg_popularity_by_genre.dat** with the genre name in quotes followed by its average. It then writes a gnuplot script **avg_popularity_by_genre.gp** that sets the chart to a wider 1400×700 size to fit all 114 genre labels, rotates the x-axis labels by -60 degrees so they don't overlap, sets the y-axis range from 0 to 70, and plots the data as a histogram. Finally, **system()** calls gnuplot to produce **avg_popularity_by_genre.png**.

```

166 double correlation(const vector<double>& x, const vector<double>& y) {
167     int n = x.size();
168     double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, sumY2 = 0;
169
170     for (int i = 0; i < n; i++) {
171         sumX += x[i];
172         sumY += y[i];
173         sumXY += x[i] * y[i];
174         sumX2 += x[i] * x[i];
175         sumY2 += y[i] * y[i];
176     }
177
178     double numerator = n * sumXY - sumX * sumY;
179     double denominator = sqrt((n * sumX2 - sumX * sumX) *
180                               (n * sumY2 - sumY * sumY));
181
182     if (denominator == 0) return 0;
183     return numerator / denominator;
184 }

```

This function compares two **vector<double>** parameters **energy** and **popularity**. It loops through both vectors simultaneously, accumulating five running totals: **sumX**, **sumY**, **sumXY**, **sumX2**, and **sumY2**, which are the building blocks of the **r** value formula. After the loop, it computes the numerator and denominator separately. The denominator uses **sqrt()** from **<cmath>** to finish the calculation. A guard checks if the denominator is 0 to avoid dividing by zero, returning 0 in that case. The final result is a number between -1 and +1 (**r** value) representing how strongly the two features are related. This function is reused nine times in **writeCorrelationWithPopularity**, once for each audio feature paired against popularity.

```

186 void writeCorrelationWithPopularity(const vector<Track>& tracks) {
187     vector<double> popularity, danceability, energy, loudness, speechiness;
188     vector<double> acousticness, instrumentalness, liveness, valence, tempo;
189
190     for (const auto& t : tracks) {
191         popularity.push_back(t.popularity);
192         danceability.push_back(t.danceability);
193         energy.push_back(t.energy);
194         loudness.push_back(t.loudness);
195         speechiness.push_back(t.speechiness);
196         acousticness.push_back(t.acousticness);
197         instrumentalness.push_back(t.instrumentalness);
198         liveness.push_back(t.liveness);
199         valence.push_back(t.valence);
200         tempo.push_back(t.tempo);
201     }
202
203     ofstream out("correlation_with_popularity.dat");
204     out << "\\danceability\\" << correlation(danceability, popularity) << "\\n";
205     out << "\\energy\\" << correlation(energy, popularity) << "\\n";
206     out << "\\loudness\\" << correlation(loudness, popularity) << "\\n";
207     out << "\\speechiness\\" << correlation(speechiness, popularity) << "\\n";
208     out << "\\acousticness\\" << correlation(acousticness, popularity) << "\\n";
209     out << "\\instrumentalness\\" << correlation(instrumentalness, popularity) << "\\n";
210     out << "\\liveness\\" << correlation(liveness, popularity) << "\\n";
211     out << "\\valence\\" << correlation(valence, popularity) << "\\n";
212     out << "\\tempo\\" << correlation(tempo, popularity) << "\\n";
213     out.close();
214
215     ofstream gp("correlation_with_popularity.gp");
216     gp << "set terminal png size 1000,600\\n";
217     gp << "set output 'correlation_with_popularity.png\\n";
218     gp << "set title 'Correlation of Audio Features with Popularity\\n";
219     gp << "set xlabel 'Feature\\n";
220     gp << "set ylabel 'Correlation\\n";
221     gp << "set style data histograms\\n";
222     gp << "set style fill solid\\n";
223     gp << "set xtics rotate by -30\\n";
224     gp << "set yrange [-0.15:0.1]\\n";
225     gp << "plot 'correlation_with_popularity.dat' using 2:xtic(1) title 'Correlation\\n";
226     gp.close();
227
228     system("/opt/homebrew/bin/gnuplot correlation_with_popularity.gp");
229 }

```

This function first declares ten separate **vector<double>** variables: One for popularity and one for each of the nine audio features (danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo). It then loops through every track and uses **push_back** to fill each vector with the corresponding field value, so by the end of the loop each vector holds all 114,000 values for that feature. It then calls the correlation function nine times, once per feature paired against popularity, and writes each result to **correlation_with_popularity.dat** with the feature name in quotes followed by its r value. The gnuplot script **correlation_with_popularity.gp** is then written, setting the y-axis range from -0.15 to 0.1 to clearly show the small differences between features, rotating x-axis labels by -30 degrees for readability, and plotting the data as a histogram. Similar to writeAveragePopularityByGenre function, the last step is having **system()** calls gnuplot to produce **correlation_with_popularity.png**.

```

231 void writeEnergyPopularityRegression(const vector<Track>& tracks) {
232     vector<double> x, y;
233
234     for (const auto& t : tracks) {
235         x.push_back(t.energy);
236         y.push_back(t.popularity);
237     }
238
239     int n = x.size();
240     double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
241
242     for (int i = 0; i < n; i++) {
243         sumX += x[i];
244         sumY += y[i];
245         sumXY += x[i] * y[i];
246         sumX2 += x[i] * x[i];
247     }
248
249     double slope = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
250     double intercept = (sumY - slope * sumX) / n;
251
252     ofstream out("energy_vs_popularity.dat");
253     ofstream pred("predicted_vs_actual.dat");
254
255     for (int i = 0; i < n; i++) {
256         double predicted = intercept + slope * x[i];
257         out << x[i] << " " << y[i] << "\\n";
258         pred << y[i] << " " << predicted << "\\n";
259     }
260     out.close();
261     pred.close();
262
263     ofstream gp1("energy_vs_popularity.gp");
264     gp1 << "set terminal png size 900,600\\n";
265     gp1 << "set output 'energy_vs_popularity.png\\n";
266     gp1 << "set title 'Energy vs Popularity with Regression Line\\n";
267     gp1 << "set xlabel 'Energy\\n";
268     gp1 << "set ylabel 'Popularity\\n";
269     gp1 << "set xrange [0:1]\\n";
270     gp1 << "set yrange [-5:105]\\n";
271     gp1 << "f(x) = " << intercept << " + " << slope << " * x\\n";
272     gp1 << "plot 'energy_vs_popularity.dat' using 1:2 with points pointtype 7 pointsize 0.3 title 'Tracks', ";
273     gp1 << "f(x) with lines lw 2 title 'Regression Line\\n";
274     gp1.close();
275
276     system("/opt/homebrew/bin/gnuplot energy_vs_popularity.gp");
277
278     ofstream gp2("predicted_vs_actual.gp");
279     gp2 << "set terminal png size 900,600\\n";
280     gp2 << "set output 'predicted_vs_actual.png\\n";
281     gp2 << "set title 'Predicted vs Actual Popularity\\n";
282     gp2 << "set xlabel 'Actual Popularity\\n";
283     gp2 << "set ylabel 'Predicted Popularity\\n";
284     gp2 << "set xrange [-5:105]\\n";
285     gp2 << "set yrange [25:45]\\n";
286     gp2 << "plot 'predicted_vs_actual.dat' using 1:2 with points pointtype 7 pointsize 0.3 title 'Predictions', ";
287     gp2 << "y with lines lw 2 title 'Perfect Prediction\\n";
288     gp2.close();
289
290     system("/opt/homebrew/bin/gnuplot predicted_vs_actual.gp");
291 }

```

This function builds two vectors x (energy) and y (popularity) by looping through all tracks, then computes four running totals to calculate the slope and intercept of the best-fit regression line. It writes two data files: 1) **energy_vs_popularity.dat** with raw energy and popularity values, and 2) **predicted_vs_actual.dat** with each track's actual popularity paired with its predicted value from the regression formula. Two gnuplot scripts are then written and executed: one plots the scatter points with the regression line overlaid, and the other plots predicted against actual popularity with a diagonal perfect-prediction line for reference, producing both PNGs.

```

int main(int argc, const char * argv[]) {
    vector<Track> tracks = loadCSV("dataset.csv");

    cout << "Loaded " << tracks.size() << " tracks." << endl;

    for (int i = 0; i < 3 && i < (int)tracks.size(); i++) {
        cout << tracks[i].track_name << " by " << tracks[i].artists
            << " | popularity: " << tracks[i].popularity
            << " | genre: " << tracks[i].track_genre << endl;
    }

    writePopularityDistribution(tracks);
    writeAveragePopularityByGenre(tracks);
    writeCorrelationWithPopularity(tracks);
    writeEnergyPopularityRegression(tracks);

    cout << "Graphs generated successfully." << endl;

    return 0;
}

```

The main function calls **loadCSV** to read all 114,000 tracks into a **vector<Track>**, prints the total count and first three tracks as a quick verification that the data loaded correctly, then calls each of the four plotting functions in order. Once all graphs are generated it prints a confirmation message and returns 0 to signal the program finished successfully.

Conclusion

This project analyzed a Spotify dataset of 114,000 tracks across 114 genres to test the hypothesis that energy and loudness would be the strongest predictors of popularity, and that explicit tracks would score higher on average. The results largely contradicted this hypothesis. Energy showed virtually no correlation with popularity ($r \approx 0.001$), and the regression line on the scatter plot was nearly flat, confirming that a song's intensity has little bearing on how popular it becomes. The multi-feature regression model further supported this, predicting almost all tracks to fall within a narrow band around 33–35 regardless of their actual popularity score, yielding an R^2 of approximately 0.02.

The most notable finding was that genre identity appeared to matter far more than any individual audio feature. Pop-film and k-pop topped the average popularity rankings while high-energy genres like metal ranked near the bottom. Explicit content did show a slightly higher average popularity, but the difference was small enough to be inconclusive. Overall, the analysis suggests that what makes a song popular cannot be captured by its sonic properties alone, and likely depends much more heavily on factors outside the dataset such as artist recognition, release recency, and cultural context.